

Universidad del Valle de Guatemala
Facultad de Ingeniería
CC3092 — Deep Learning y Sistemas Inteligentes

Laboratorio #9

Series de tiempo y forecasting

Nicolás Concuá
Esteban Cárcamo
Guatemala, noviembre de 2026

1. Problema, datos y preparación de la serie

Pronóstico **day-ahead** (H = 24 h, emitido a las 00:00 con datos hasta las 23:00) del consumo horario en kWh de clientes de Portugal (ElectricityLoadDiagrams20112014, UCI). El consumo de una hora es la suma de los 4 registros de 15 min dividida entre 4. Entrenamiento hasta el 2014-06-30, validación jul–sep 2014 y prueba oct–dic 2014 con origen móvil diario (92 orígenes).

- **Filtro.** De 370 clientes quedan **314** sin rachas de ≥ 24 h en cero en 2012–2014. Por cómputo se usan 100 elegidos con semilla 42, los mismos para todos los modelos. Cada serie tiene 26,304 horas y el panel usado, 2.63 M observaciones.
- **Cambio de horario.** Malla regular de 24 h/día: la 1:00 de marzo (≈ 0) se interpola con las 0:00 y las 2:00, y la 1:00 de octubre (dos horas acumuladas) se divide entre 2.
- **Escalas y normalización.** El consumo medio de entrenamiento va de 21 a 11,480 kWh/h (mediana 160), una razón de **≈ 550×** (casi 3 órdenes de magnitud). Se usa z-score **por serie** con μ y σ de entrenamiento (con una normalización global los clientes pequeños quedarían en ≈ 0 y la pérdida la dominarían los grandes) y, como iteración, RevIN por ventana.
- **Covariables de calendario.** Hora, día de la semana y mes en seno/coseno, fin de semana y feriado de Portugal (paquete holidays, que incluye la suspensión de 4 feriados en 2013–2015). El 25-dic el consumo cae ~33 % y el 1-ene ~20 % frente al mismo día de la semana de las 3 semanas previas.
- **Ventanas (L = 168, H = 24, stride 1 h).** 2,169,700 de entrenamiento y 218,500 de validación y de prueba. Un Dataset de PyTorch entrega X (batch, L, 9), y (batch, 24) y el calendario futuro (batch, 24, 8).
- **Auditoría de fugas (con assert en el código).** (1) Cortes por fecha, sin barajar entre particiones; (2) μ, σ y la escala del MASE solo de entrenamiento; (3) early stopping con validación y la prueba solo en backtest; (4) la última entrada es t0 − 1 y el futuro solo trae calendario.

2. Investigación y protocolo

Herramientas. nn.LSTM/nn.GRU (input_size, hidden_size, num_layers, batch_first, dropout; salidas output = estado de la última capa en cada paso, h_n y c_n = estado final de cada capa) [1]; **nn.Conv1d** con padding causal (solo a la izquierda) y dilataciones $1\ldots2^n$, campo receptivo $1 + \Sigma(k - 1)d_i$ [2]; **nn.TransformerEncoderLayer** (d_model, nhead, dim_feedforward) con codificación posicional sumada a los embeddings [3]; **NeuralForecast/StatsForecast** (formato unique_id, ds, y; cross_validation con n_windows, step_size y refit) [4] y **Chronos** (predict_quantiles con context, prediction_length y quantile_levels) [5]. **Conceptos:** modelo local (uno por serie) vs. global (uno para todas, comparte patrones); teacher forcing (alimentar el valor real en lugar de la predicción al entrenar); RevIN (normaliza cada ventana y revierte a la salida, robusto a cambios de nivel) [6]; pinball loss (su minimizador es el cuantil τ); MASE (por serie, relativo al seasonal naive) vs. WAPE (pondera por volumen) [7].

Protocolo. Una sola función **backtest(modelo)** evalúa todo: el modelo expone **predecir(hist, t0)** y recibe una copia de solo lectura de la historia hasta las 23:00 del día anterior. Los modelos de librería se adaptan a esa interfaz (NeuralForecast se entrena una vez con datos hasta el 2014-09-30, con val_size = jul–sep, y pronostica en cada origen). Métricas en kWh: MAE, RMSE, WAPE, MAPE, MASE (escala: MAE del seasonal naive de 24 h en entrenamiento, por cliente) y, para los probabilísticos, pinball (P10/P50/P90) y cobertura P10–P90. El MAPE no explota aquí porque el cliente más pequeño consume ~21 kWh/h (0.63 % de horas con y < 1), pero con consumos ≈ 0 su denominador se dispara y lo dominan esos clientes.

3. Iteraciones de hiperparámetros (solo validación) y estrategias multi-paso

Tabla 1. Iteraciones (H = 24). En negrita, las elegidas con validación. Base: L = 168, 30 meses, normalización por serie, lr = 10^{-3}, batch 512.

Iteración	L	Meses	Norm.	Oculto	Drop.	lr	Batch	Params	Épocas (mejor)	MAE val	MASE val	s/época	Total (s)	GPU (MB)
MLP	168	30	serie	512	0.1	0.001	512	322 k	26 (21)	32.18	0.841	0.64	17	32
MLP-L48	48	30	serie	512	0.1	0.001	512	261 k	31 (26)	35.07	0.917	0.71	22	20
MLP-L336	336	30	serie	512	0.1	0.001	512	408 k	14 (9)	31.68	0.835	1.07	16	47
MLP-m3	168	3	serie	512	0.1	0.001	512	322 k	6 (1)	39.77	1.037	0.77	5	31
MLP-m12	168	12	serie	512	0.1	0.001	512	322 k	18 (13)	31.97	0.842	0.92	17	31
MLP-revin	168	30	ventana	512	0.1	0.001	512	322 k	25 (20)	31.26	0.826	1.00	26	43
MLP-h1024	168	30	serie	1024	0.1	0.001	512	907 k	30 (25)	31.54	0.826	0.94	29	49
MLP-drop0.3	168	30	serie	512	0.3	0.001	512	322 k	19 (14)	32.47	0.861	1.04	20	31
MLP-lr3e-4	**168**	**30**	**serie**	**512**	**0.1**	**0.0003**	**512**	**322 k**	**33 (28)**	**31.17**	**0.823**	**0.88**	**30**	**31**
MLP-b128	168	30	serie	512	0.1	0.001	128	322 k	25 (20)	32.94	0.853	2.03	51	30
LSTM-AR	168	30	serie	64	0.1	0.001	512	53 k	40 (38)	35.94	0.940	4.05	171	583
LSTM-AR-e80	**168**	**30**	**serie**	**64**	**0.1**	**0.001**	**512**	**53 k**	**80 (76)**	**34.64**	**0.903**	**4.68**	**397**	**579**
DLinear-m3	168	3	serie	—	—	0.005	512	8 k	17 (12)	35.31	0.907	0.99	17	21
LSTM-AR-m3	168	3	serie	64	0.1	0.001	512	53 k	15 (10)	49.90	1.315	4.73	76	579

Inferencia sobre los 92 orígenes: < 1 s en los MLP y ~2 s en la LSTM-AR. Parámetros totales = entrenables.

Lectura. L = 48 empeora (no ve la semana previa); con 3 meses de historia el MLP sobreajusta desde la época 1; RevIN (0.826) y bajar el lr (0.823) son lo que más ayuda, y la LSTM-AR con 40 épocas estaba subentrenada (0.940 → 0.903 con 80). **¿Menor pérdida de entrenamiento implica mejor MASE de validación?** No siempre: en el MLP la correlación es alta (Spearman 0.93), pero la variante de 1,024 unidades tiene la menor pérdida de entrenamiento sin ser la mejor, y con 3 meses la pérdida de entrenamiento baja mientras la de validación sube (Figura 1).

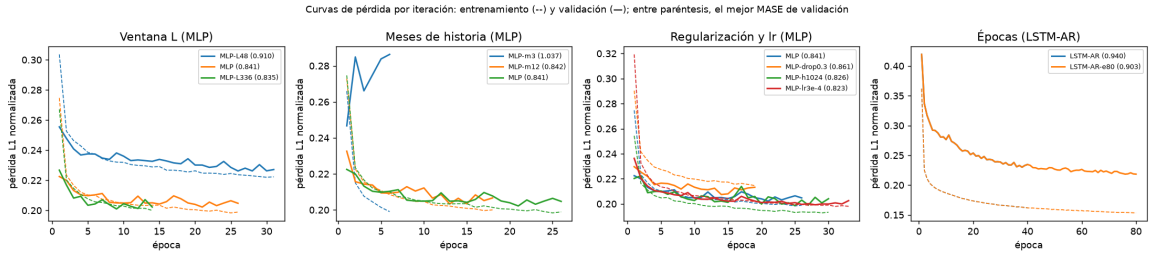


Figura 1. Pérdida de entrenamiento (--) y validación (—) por iteración; entre paréntesis, el mejor MASE de validación.

Estrategias multi-paso (LSTM, 40 épocas). La autorregresiva es la mejor en h = 1 (25.9 kWh), pero su error crece más (MAE h24/h1 = 1.78) que el de la multi-salida (1.44) y el del encoder–decoder (1.58): hay acumulación de error, moderada a 24 h y dominante a 168 h (MASE de la LSTM-AR 1.15 y del seq2seq 1.55). En el total de las 24 h ganó la autorregresiva (MASE 0.910 frente a 0.947 y 0.946). La Figura 2 muestra el MAE por paso de todos los modelos.

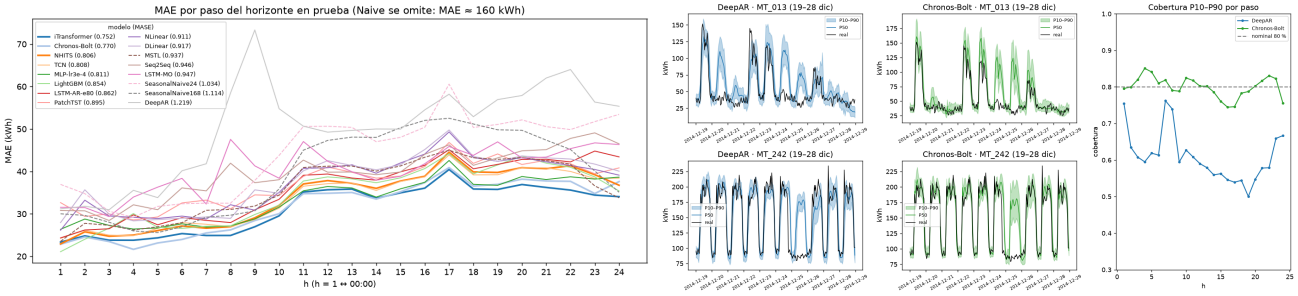


Figura 2. Izquierda: MAE por paso del horizonte de todos los modelos (h = 1 ↔ 00:00). Derecha: bandas P10–P90 de DeepAR y Chronos en Navidad y cobertura por paso (sección 4).

4. Comparación de todos los modelos (prueba, 92 orígenes, 100 clientes)

Tabla 2. Mejor configuración de cada modelo, ordenada por MASE (H = 24).

Modelo	Familia	L	Estr.	Cov.	Params tot.	Params entr.	Entren. (s)	Infer. (s)	Hw.	MAE	RMSE	WAPE	MAPE	MASE	MASE H168	MASE Navidad	Pin-ball	Cob. 80
iTransformer	**Atención**	**168**	**MS**	**no**	**1.21 M**	**1.21 M**	**854**	**2.0**	**GPU***	**31.3**	**163**	**6.1%**	**7.9%**	**0.752**	—	**1.252**	—	—
Chronos-Bolt	**Fundacional**	**2048**	**MS**	**no**	**47.72 M**	**0**	—	**321.1**	**CPU**	**31.7**	**200**	**6.2%**	**8.1%**	**0.770**	**0.930**	**1.443**	**0.257**	**80.0%**
N-HiTS	Resid./conv.	168	MS	sí	3.80 M	3.80 M	149	5.2	GPU*	33.9	207	6.6%	8.5%	0.806	—	1.647	—	—
TCN	Resid./conv.	168	MS	no	251 k	251 k	620	8.8	GPU*	33.9	190	6.6%	8.8%	0.808	—	1.544	—	—
MLP-lr3e-4	MLP/lineal	168	MS	sí	322 k	322 k	30	0.0	GPU*	33.4	166	6.5%	8.8%	0.811	—	1.338	—	—
LightGBM	Árboles	168	AR	sí	381 k	381 k	158	23.6	CPU	34.5	169	6.7%	9.0%	0.854	1.064	1.505	—	—
LSTM-AR-e80	Recurrente	168	AR	sí	53 k	53 k	397	1.9	GPU*	36.0	177	7.0%	9.1%	0.862	1.152	1.409	—	—
PatchTST	Atención	168	MS	no	445 k	445 k	113	5.5	GPU*	37.2	183	7.2%	10.2%	0.895	—	1.675	—	—
NLinear	MLP/lineal	168	MS	no	4 k	4 k	7	0.0	GPU*	37.2	186	7.2%	10.5%	0.911	—	1.813	—	—
DLinear	MLP/lineal	168	MS	no	8 k	8 k	5	0.0	GPU*	37.8	188	7.3%	10.7%	0.917	—	1.834	—	—
MSTL	Baseline	672	—	no	0	0	122	0.0	CPU	36.0	183	7.0%	10.4%	0.937	1.113	1.970	—	—
Seq2Seq	Enc.–dec.	168	ED	sí	105 k	105 k	326	2.1	GPU*	39.6	186	7.7%	10.4%	0.946	1.554	1.605	—	—
LSTM-MO	Recurrente	168	MS	sí	59 k	59 k	164	1.6	GPU*	40.4	193	7.8%	10.5%	0.947	—	1.650	—	—
SeasonalNaive24	Baseline	24	—	no	0	0	0.4	0.0	CPU	44.1	232	8.6%	11.8%	1.034	1.188	2.168	—	—
SeasonalNaive168	Baseline	168	—	no	0	0	0.4	0.0	CPU	39.7	182	7.7%	11.2%	1.114	1.115	1.602	—	—
DeepAR	Probabilístico	168	AR	sí	204 k	204 k	2,011	37.0	GPU*	50.0	218	9.7%	13.9%	1.219	—	1.815	0.417	60.9%
Naive	Baseline	1	—	no	0	0	0.4	0.0	CPU	158.6	505	30.8%	40.0%	3.989	4.090	3.906	—	—

*MS = multi-salida, AR = autorregresiva, ED = encoder–decoder. MASE H168: 13 orígenes semanales, solo modelos que pronostican 168 h sin reentrenarse. Pinball dividido por la escala MASE. Params de LightGBM = hojas totales. *Entrenado en GPU (RTX 4050 Laptop, 6 GB); inferencia final en CPU (i9-13980HX).*

Hardware y parámetros. La ejecución final del notebook se hizo en CPU con los pesos en caché porque la laptop se apagaba por temperatura con la GPU a plena carga; por eso los tiempos de inferencia son de CPU. Chronos no se entrena (su tiempo es solo de inferencia, en CPU, y sus 47.7 M parámetros están congelados); en el resto los parámetros entrenables coinciden con los totales, salvo diferencias de < 10 en PatchTST y DeepAR.

Error por cliente. El MAE global lo dominan los clientes grandes: con iTransformer el MAE por cliente tiene mediana **10.0 kWh** (P10–P90: 3.1–41.9) y RMSE mediano 15.7 kWh, mientras que el cliente más grande (MT_279, ~11,400 kWh/h) tiene MAE 717 y RMSE 1,383. En términos relativos el error mediano por cliente es 6.2 % de su consumo (seasonal naive 24 h: 13.1 kWh y 8.3 %). Por eso el MASE se promedia por cliente y se reporta junto al WAPE.

Resultados probabilísticos. Chronos-Bolt está bien calibrado (cobertura P10–P90 de 80.0 %, pinball 0.257) y abre el intervalo el 25-dic, cuando no sabe si será feriado. DeepAR (34 min de entrenamiento) queda peor que el seasonal naive (MASE 1.219), con intervalos demasiado estrechos (61 % de cobertura, que cae con el horizonte). **Trampa encontrada:** con scaler_type="standard", NeuralForecast normaliza también las covariables con la media y la desviación de la ventana de entrada (+10⁻⁶); el mes (o el feriado) es constante en una semana y, al cambiar el primer día del mes, su valor futuro se multiplica por ~10⁶. Por eso el TCN con calendario divergía el 1-ago (se eligió con validación la variante sin covariables) y N-HiTS explotaba el 1-dic; reentrenado solo con hora, día y fin de semana pasó de 0.915 a 0.806.

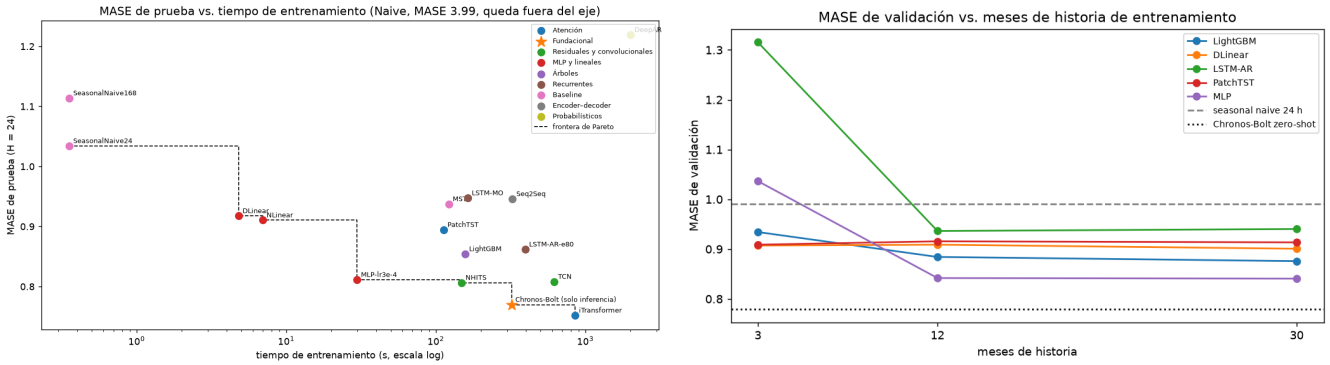


Figura 3. Izquierda: MASE de prueba vs. tiempo de entrenamiento (log) y frontera de Pareto (Chronos, en su tiempo de inferencia en CPU). Derecha: MASE de validación vs. meses de historia; líneas: seasonal naive y Chronos.

5. Análisis y conclusiones

- **Ganador.** iTransformer: MASE 0.752 contra 1.034 del seasonal naive (27 % menos; MAE 31.3 vs. 44.1 kWh). Es el único que mezcla clientes (cada serie es un token). Cuesta 14 min de GPU, pero Chronos-Bolt queda a 0.018 sin entrenar y el MLP afinado (0.811) se entrena en 30 s: la mejora solo justifica su mantenimiento si cada punto de error vale mucho (compras en el mercado diario).
- **DLinear vs. Transformers.** Con H = 24, DLinear (0.917) y NLinear (0.911) casi empatan con PatchTST (0.895) con 55 a 110 veces menos parámetros; no compiten con iTransformer, Chronos, N-HiTS (0.806) ni el TCN. A 168 h no entrenamos versiones directas, así que no lo afirmamos a ese horizonte.
- **Error por paso.** Con el origen a las 00:00, h coincide con la hora del día: en todas las familias el error sigue la forma del consumo (máximo de 10:00 a 17:00) más que una tendencia con el horizonte.
- **L, normalización e historia.** Hace falta ver al menos una semana (L ≥ 168); la normalización por serie es imprescindible y RevIN aporta algo más. Con 3 meses, el MLP y la LSTM pierden contra el seasonal naive; desde 12 meses el MLP supera a LightGBM (0.842 vs. 0.884) y a los estadísticos, y más historia casi no ayuda. La LSTM-AR nunca supera a LightGBM.
- **Global vs. local.** Con el MLP (mejor modelo propio), el global gana por poco en los 10 clientes (0.852 vs. 0.861; mejor en 5 de 10) con un solo modelo; se benefician más los clientes cuyo modelo local sobreajusta pronto (MT_174: +11.5 %).
- **Chronos zero-shot.** Segundo en MASE, el mejor a 168 h (0.930) y el mejor calibrado. Lo usaríamos con clientes nuevos o con poca historia, sin infraestructura de entrenamiento o cuando se necesitan intervalos.
- **Navidad.** La semana del 22 al 28 de diciembre es la peor para todos los modelos (MASE de 1.25 a 2.17 sin contar el Naive, que llega a 3.91). El calendario ayuda (MLP 1.34 y LSTM-AR 1.41 frente a DLinear 1.83 y MSTL 1.97), pero el mejor es iTransformer (1.25), que no lo usa y detecta el cambio de régimen al ver a todos los clientes a la vez.
- **Reentrenamiento y semillas.** Se hicieron con el MLP, el mejor modelo de deep learning propio, en lugar de iTransformer: cada entrenamiento de iTransformer cuesta ~14 min de GPU y repetirlo (3 reentrenamientos + 3 semillas) no era viable en la laptop por temperatura. Reentrenar cada mes no cambia el error (MLP 0.8113 → 0.8116; MSTL 0.939 ajustado una vez → 0.938 mensual y 0.937 diario). El MLP con 3 semillas da 0.813 ± 0.008: diferencias menores a ~0.01 no son concluyentes; las diferencias entre familias (≥ 0.05) sí.
- **Conclusión.** Lo que más reduce el error es compartir información entre clientes (modelo global, iTransformer) o entre dominios (Chronos); la complejidad por sí sola no garantiza nada (DeepAR), y el preprocesamiento pesa tanto como la arquitectura (N-HiTS: 0.915 → 0.806 al corregir las covariables). Recomendamos Chronos-Bolt como punto de partida y el MLP global o iTransformer si el valor del error lo justifica.

Referencias

[1] PyTorch Documentation. torch.nn (LSTM, GRU, Conv1d, TransformerEncoderLayer). https://pytorch.org/docs/stable/nn.html

[2] Bai, S., Kolter, J. Z. y Koltun, V. (2018). An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv:1803.01271.

[3] Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS.

[4] Nixtla. StatsForecast y NeuralForecast documentation. https://nixtlaverse.nixtla.io

[5] Ansari, A. F. et al. (2024). Chronos: Learning the Language of Time Series. TMLR.

[6] Kim, T. et al. (2022). Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift. ICLR.

[7] Hyndman, R. J. y Koehler, A. B. (2006). Another look at measures of forecast accuracy. International Journal of Forecasting, 22(4).

Enlace al repositorio: https://github.com/nicoCT04/Lab9-DL